

- **Main: Reads filenames from command-line and starts the planner**

```
if args.length != 3:
    print usage
    exit
planner = new FlightPlanner(flightsFile, requestsFile, outputFile)
planner.run()
```

- **FlightPlanner: Loads input files, finds routes, sorts them, and writes output**

```
load flight data into FlightGraph
load request list into RequestManager
```

```
create Pathfinder using FlightGraph
create OutputWriter (output file)
```

```
flightNum = 1
```

```
for each request:
    writer.writeHeader(flightNum, request)
```

```
    paths = pathFinder.findAllPaths(origin, destination)
```

```
    if paths empty:
        writer.writeNoPlan()
        writer.writeBlank()
        flightNum++
        continue
```

```
    heapSort(paths, request.sortType)
```

```
    write top 3 paths using writer
```

```
    writer.writeBlank()
    flightNum++
```

```
writer.close()
```

- **FlightGraph: Stores flight connections using an adjacency list and loads the graph from OrgDestData file**

```
initialize map city → list of edges
```

```
read number of records
```

```
for each record:
    skip blank lines
    parse origin, dest, cost, time
```

```
    add edge origin → dest
    add edge dest → origin
```

- **RequestManager: Loads all requested flight searches and stores them in a list**  
read number of requests

```
for each request:  
    skip blank lines  
    parse origin, destination, sortChar  
  
    create Request and add to list
```

- **PathFinder: Performs iterative backtracking DFS to generate all the possible paths between two cities**

```
stack.push(initial frame with origin)
```

```
while stack not empty:  
    frame = stack.pop()
```

```
    if frame.city == destination:  
        convert to Path and store  
        continue
```

```
    for each outgoing edge from frame.city:  
        if nextCity not visited:  
            create new frame with updated path, cost, time  
            push onto stack
```

```
return list of all complete Paths
```

- **HeapSort: Sorts all found paths using HeapSort based on cost or time**  
build max heap based on chosen metric

```
for end from last to first:  
    swap root with end  
    heapify reduced heap
```